

Stock Buy-Sell Recommendation using Machine Learning

Gautam Tevatia, Kavita Ganesan, Roshan Herbert
University of Southern California
Computer Science Department
Los Angeles, CA, 90089
{[ganesan](#), [raherber](#), [tevatia](#)}@usc.edu

Abstract

This project attempts to compare and evaluate stock buy-sell recommendations using machine learning algorithms namely Multilayer Neural Networks and Support Vector Machines. The predictions made using these two learners are then compared with recommendations by analysts. The comparisons were made to determine if machine learning algorithms can predict as well as human analysts. Our findings suggest that our Support Vector Machine prediction model is comparable to predictions made by analysts, but the Neural Networks model performs rather poorly. In terms of profitability however, the Neural Networks model yields the highest returns, followed by the Support Vector Machine model and then the human analysts. This shows that though the learners do not completely agree with the true predictions, these learners yield higher returns, which is a very positive sign and calls for more research in this area.

1. Introduction

The idea of stock buy-sell predictions is simply to maximize your trading profit by predicting the best BUY or SELL moments for stocks. Stock predictions have been traditionally performed by financial analysts. These analysts typically make forecasts about companies' future earnings, revenues, growth rates, and stock price (price target). They also make recommendations regarding buying, selling, or holding a particular stock. Over the recent years, researches have been attempting to use machines to do jobs that have been traditionally done by these analysts.

Recommendations by analysts are typically made using trend information from historical financial data. When human analysts can make predictions using such trend information, why would machines not be able to do the same with the very same historical data? This has been the underlying motivation for the increase in research interest in this area.

In this paper, we aim to determine if machine learning algorithms can make buy-sell predictions similar to financial analysts. We also seek to get an understanding of how well the learners perform in terms of profitability in contrast to analysts.

In the next section, we start by discussing related work in this area. This is followed by the motivation for doing this project. We then describe the dataset used in this project and then provide some insights into the selection of the features. This is followed by some discussion on the training phase with some description on the how the best models were determined. An evaluation of the performance of the prediction models system is then described. We conclude with a discussion on the overall performance of our prediction models and provide suggestions as to how these models can be further improved.

2. Related Work and Motivation

There has been considerable amount of research work in using machine learning algorithms for financial predictions. Researchers in this domain have been predominantly using Neural Networks as the learning algorithm for it is a very flexible and reliable algorithm.

The work by Karl Nygren in his Master's Theses [8] focuses on the use of Neural Networks to make prediction of stock prices. Karl's work is very mathematically inclined and the main emphasis is on the statistical aspects of using Neural Networks in this domain.

Ramon Lawrence, from the University of Manitoba, wrote an article on the application of weighted Neural Networks in forecasting stock market prices [9]. Lawrence, in his research work, discusses common market analytics techniques and contrasts them with Neural Networks.

Jingtao Yao, in one of his papers has described how to build a model that uses Neural Networks for financial

forecasting [10]. He discusses several approaches to data pre-processing and data sampling which we have used as guidelines to pre-process our data.

3. Motivation

Our work is similar to [9] in that, instead of evaluating the use of machine learning algorithms for stock price predictions, we attempt to use machine learning algorithms to make stock buy-sell recommendations. Apart from that, we also differ in our approach where we do not compare different variations of Neural Networks, but we use different learners and compare the performance of these learners against predictions made by human analysts. With this, we get to evaluate to what extent machines predict as well as humans.

Despite the fact that there are commercial tools like the Stock NeuroMaster [1] that do buy-sell predictions using Neural Networks, the absence of relevant research papers in this area makes it a hard task to know how well these tools could possibly perform in making such predictions. Thus in this paper, we attempt to provide a general idea of how learning algorithms apart from Neural Networks perform in this domain in comparison to financial analysts.

4. Dataset

The crucial part of the recommendation system is obtaining data in the requisite format. We obtained our data from several sources. This includes the Wharton Research Data Services Database (WRDS) [2] Yahoo!Finance[3] and MarketWatch[5].

WRDS is a warehouse of financial, accounting, banking and economics databases accessible through a Web-based interface. It is a free database for researches. Within the WRDS database itself there are several different datasets. We specifically use the Compustat and the I/B/E/S dataset in WRDS.

The Compustat dataset in general provides the annual and quarterly income statement, balance sheet, statement of cash flows, and supplemental data items on most publicly held companies. We use this dataset to obtain data such as the Earnings per Share (EPS), current liability, current asset, total liability and total asset.

I/B/E/S dataset on the other hand provides other information like the return on equity (ROE), price to earnings growth, company income and company sales. Apart from that, this dataset also provides historical stock buy-sell predictions. These predictions are predictions

used as labels to our training examples. Figure 1 shows an example of how the buy-sell predictions look in I/B/E/S.

Historical stock prices were not available in the WRDS database and hence this was obtained from Yahoo!Finance. Figure 2 shows an example of data provided by Yahoo!Finance.

OFTIC	RECDATS	CNAME	IRECCD	ITEXT
IBM	09MAR2004	INTL BUS MACH	2	BUY
IBM	24MAR2004	INTL BUS MACH	1	STRONG BUY
IBM	13APR2004	INTL BUS MACH	3	HOLD
IBM	16APR2004	INTL BUS MACH	1	STRONG BUY
IBM	02JUN2004	INTL BUS MACH	2	BUY
IBM	04JUN2004	INTL BUS MACH	2	BUY
IBM	14JUN2004	INTL BUS MACH	3	HOLD
IBM	17JUN2004	INTL BUS MACH	2	BUY
IBM	13JUL2004	INTL BUS MACH	2	BUY
IBM	28JUL2004	INTL BUS MACH	2	BUY

Figure 1: Sample predictions from I/B/E/S

MarketWatch is simply a provider of business news, financial information and analytical tools. This source was used to obtain predictions made by analysts. The purpose of these predictions is for evaluation and this will be explained in detail in section 7.

SET DATE RANGE

Start Date:

Jan

2

2002

Eg. Jan 1, 2003

End Date:

Apr

9

2006

Get Prices

☒ Daily
 ☐ Weekly
 ☐ Monthly
 ☐ Dividends Only

First | Prev | [Next](#) | Last

PRICES

Date	Open	High	Low	Close	Volume	Adj Close*
7-Apr-06	83.60	83.80	82.44	82.48	5,337,500	82.48
6-Apr-06	83.70	84.32	83.36	83.81	4,944,100	83.81
5-Apr-06	83.33	84.45	83.32	84.17	5,523,900	84.17
4-Apr-06	83.05	83.78	82.80	83.45	3,853,000	83.45
3-Apr-06	82.72	83.59	82.65	83.06	4,121,600	83.06

Figure 2: Example of daily historical stock prices from Yahoo!Finance

We obtained data from years 2001 to 2005 for twenty fortune 500 companies within the semiconductor industry. We limited our focus to one industry as different industries have different prediction models. We cannot have a dataset which includes different kinds of industries as we are essentially looking to narrow down on companies which at least have a similar pattern for growth rate, return and risk. However if there is a need for a mix of industries, then choosing the industries from the Dow Jones Industrials would make a good dataset. Since our model uses quarterly predictions, our training data size was almost 400 (20*4(quarters)*5(years)) rows.

5. Feature Selection and Pre-processing

The number of features for our analysis can be significant in number. However, we selected only the features of maximum pertinence and use. Having spent a considerable amount of time researching the best features to be used in this domain, we obtained a total of seven key features. The selected features basically provide discriminating information that can help the machine learning algorithms make the expected predictions.

In the following section, we provide a brief description on the features used in this project and how these features were derived if not obtained directly from the data sources. Next, the labels used for training the learners is described and this is followed a discussion on how missing values were handled. We then discuss the pre-processing tasks involved in getting our training examples in the format that our model supports.

5.1 Features

We use seven important features to train the learners to make buy-sell predictions. The first and most important feature used is the price to earnings (P/E) ratio. This ratio is a useful metric for evaluating the relative attractiveness of a company's stock price [3]. It can help differentiate between a less-than-perfect stock selling at a high price and a great company which may have fallen out of favor and is selling for a fraction of what it is truly worth. This ratio is not provided directly by the data sources and has been computed using the following formula:

P/E = Average stock close price in quarter/Quarterly EPS

The second feature used is the return on equity (ROE). Return on equity reveals how much profit a company has earned in comparison to the total amount of shareholder equity found on the balance sheet [4]. Next, is the ratio of the number of ups of the stock price within a given quarter. This feature requires computing the total number of ups ([close price-open price] >0) over the total number of days for which the stock price was available within a quarter. This feature essentially gives an idea about the general trend of the stock in question. The reason for using this as compared to the stock price itself is because stock prices generally vary with time, sector and company and may possibly affect the prediction model.

The sales growth and income growth depict the potential of the company to re-invest in future. The income of the company is very important in determining the cash flow strength of the company. A consistent growth in sales is a good sign and depicts a growing industry. Sales growth

and income growth could not be obtained directly from the data sources and have been computed as follows:

Sales growth= [sales in current quarter-sales in previous quarter]/sales in previous quarter

Income growth= [income in current quarter-income in previous quarter]/income in previous quarter

Apart from the profitability features for a company we have tried to include a feature representative of the liquidity of a company and another feature that gives an idea about the solvency of a company. Such diverse selection helps us approach the learning problem taking into account some of the fundamental features which are used by analysts to make predictions. The current-ratio suggests whether a company has the ability to pay off its current liabilities or not. Analysts view a current ratio of more than one to be an indication that the firm can cover its current liabilities. This is computed as following:

Current Ratio=current asset/current liability

The debt to total assets ratio also known as total liability to asset ratio helps in determining the solvency of the company. The higher the debt to total assets, the greater the risk that the company may not be able to pay off its debts. This ratio again is a derived attribute.

Feature	Derived?	Formula	Source
P/E	Yes	Avg(Close price in quarter)/ quarterly EPS)	Close Price – Yahoo!Finance EPS - Compustat
Up/Down ratio of stock price	Yes	Sum((Close price–Open Price)>0)/# days with stock price	Open price - Yahoo!Finance Close price - Yahoo!Finance
ROE	No		I/B/E/S
Income growth	Yes	(Income in current quarter-Income in previous quarter)/ Income in previous quarter	I/B/E/S
Sales growth	Yes	(Sales in current quarter-Sales in previous quarter)/ Sales in previous quarter	I/B/E/S
Current ratio	Yes	Current asset (quarterly)/ Current liability (quarterly)	Current Asset – Compustat Current Liability- Compustat
Debt/ total asset ratio	Yes	Total liability (quarterly)/ Total asset (quarterly)	Total Asset – Compustat Total Liability- Compustat

Table 1: Summary of features

Table 1 provides a summary of the features, the sources the features come from and how the features have been computed if they were derived attributes.

5.2 Labels

The labels used for training is simply to indicate that for a given set of features, what the prediction could be. In the original dataset (I/B/E/S), there are five different possible labels. They are “Underperform”, “Sell”, “Hold”, “Buy”, and “Strong Buy”. To reduce the complexity of the prediction model, we made “Buy” and “Strong Buy” as just “Buy” and “Sell” and “Underperform” as just “Sell”. This leaves us with three different possible predictions.

5.3 Handling Missing Values

There were several points at which missing values were detected. First, for some of the quarters within a year, prediction labels were not available. For such cases, we analyze the majority predictions in the other quarters of the year and use the prediction label with the highest vote. Say for the first three quarters in a given year, the prediction label was a “Buy” and for the fourth quarter, the prediction label is missing. Then, the value to fill the missing value will be a “Buy”.

Apart from missing labels, some of the EPS values used to calculate the P/E ratio were missing from the Compustat dataset. So, for these cases, we used the EPS values which were provided by the I/B/E/S source.

5.4 Data pre-processing

The prediction labels provided by the I/B/E/S data source has a monthly periodicity. After having analyzed the monthly prediction labels closely, we find that the prediction labels and the corresponding feature values within a quarter changes very infrequently. Thus, using monthly examples would have resulted in redundant training examples, which deceives the purpose of machine learning. From our analysis of the data, we came to a conclusion that it would be best to have a quarterly prediction model.

Some of the features from the data sources used were only available daily or monthly and requires some level of pre-processing. This pre-processing predominantly involves obtaining the average values of the features over a quarter.

As for the labels, we get the labels with the majority vote within a quarter. Say we have ten labels with a “hold” and two labels with a “buy” within a quarter of a year, the final label will be a “hold”.

Apart from getting the feature values in their quarterly form, we also had to combine features and labels coming from different sources to be able to get a unified example for training. This unification was done using the ticker symbol of companies, quarter of year and year information of each source. Figure 3 shows the pre-processing tasks involved in building the training examples. The “q” in this diagram means quarterly, “m” means monthly and “d” means daily.

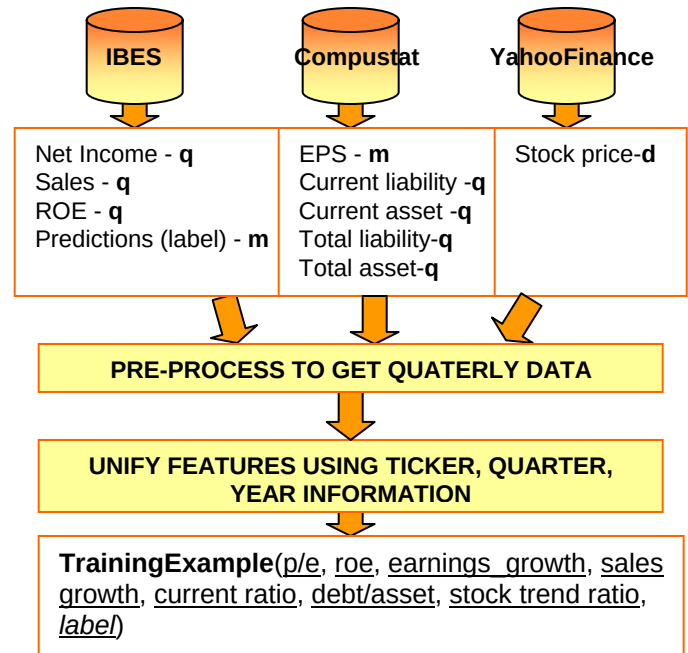


Figure 3: Pre-processing tasks

6. Training

In this project, the two learners that we use are Support Vector Machine and Multilayer Neural Networks. The rationale for using Neural Networks is one, it is the most common learner used in the financial domain. Second, it is known that Neural Networks can handle linearly inseparable data which is the characteristic of the data that we have. For the same reason, we decided to also experiment with Support Vector Machines. Support Vector Machines have been rarely used in the financial domain and experimenting with this learner can help evaluate and compare the performance of the two learners.

The training starts with identifying the best prediction model for the two learners that we use. The best model simply means the model with the highest accuracy. We do this by varying the parameter settings for the learners and

by analyzing the average mean absolute errors for the different parameter settings.

Since we deal with data that is temporally dependant, to obtain the error rates, we use the temporal sliding window approach for the different models. Temporal sliding window simply means that the training will be done on data that is chronologically before the data that we test on. For example, for a given model, say we train on data from year 2001. We then test the model on data from year 2002, and we note down the mean absolute error. Then we train on data from year 2002 and test on data from year 2003, and note down the mean absolute error. This is repeated until we complete the cycle of training and testing from years 2001 to 2005. The mean absolute errors obtained are then averaged over the number of years for which we did the testing. We then move on to the next combination of parameter settings and do the same as before. The best model will be the model with the lowest average error and this model will be used for the actual training.

Training Data	Test Data				
		2002	2003	2004	2005
	2001	0.389			
	2002		0.372		
	2003			0.365	
	2004				0.381
Parameters: learningRate=0.01, momentum=0.2, hiddenLayer=1, normalizeAttributes=true, randomSeed=0, epoch=1000					

Table2: Mean absolute errors for best Neural Network model

Training Data	Test Data				
		2002	2003	2004	2005
	2001	0.357			
	2002		0.339		
	2003			0.366	
	2004				0.338
Parameters: Kernel=polynomial, C=10, Exponent=3, Normalized=Yes					

Table 3: Mean absolute errors for best Support Vector Machine model

For Neural Networks, the learning rate, momentum, number of hidden layers, seed value and the number of epochs were varied. For Support Vector Machines, we use the polynomial kernel and vary the exponent and the size of the margin (C value). Having used different combinations of parameter settings, the lowest average mean absolute error obtained for Neural Networks is 0.38 while for Support Vector Machine, the lowest error value is 0.35. Tables 2 and 3 show the mean absolute errors

obtained for the “best models” and the corresponding parameter settings used.

7. Evaluation and Results

Our evaluation in this paper is three folded. We first determine the difference in error between the two learning methods used (Support Vector Machine and Neural Networks). This is done to see if the two learners have the same error so that we could compare the learners. We then compare how the analysts do in making predictions as compared to the two learners. This is to see if the two learners do just as well as the analyst or perform worse than the analysts. Next, we analyze to see if one would have made money if they were to buy and sell stocks based on predictions of our system or based on the recommendations of the analysts.

The next section describes the paired t-test approach used to determine the likelihood of the two learners having the same errors. The section after this, describes how performance of human analysts and the learners are compared and provides some insights into the results obtained. Lastly, we evaluate if the predictions made by analysts bring more profits or do we make more profits with the predictions of the learners.

7.1 One-tailed paired t-test

The statistical significance of the difference in classification errors is tested using the paired t-test. This test is important to be able to compare the two learners that we are using for making buy-sell predictions.

The paired t-test is in general used for comparing the means of error distributions of the classification algorithms. It measures the likelihood that the classification algorithms have the same overall performance. For a fixed significance level ‘a’, If a difference is significant at the a% level, there is a (100-a) % chance that there really is a difference

In this project, the one-tailed paired t-test has been used to compare the performance of the Multilayer Neural Networks model and the Support Vector Machine model to see if one performs better than the other. For each dataset (data from years 2002 to 2005), the learners were run and the four pairs of mean absolute errors were obtained. The details on how the errors were obtained have been explained in section 6. The paired t-test was performed using a significance level of 0.05. The results obtained for the paired t-tests are as shown in table 4.

With the one tailed paired t-test results obtained, it can be

inferred that the means of the error distributions of these algorithms are significantly different. Based on these values, it can be concluded that with a probability of 96.6% the Support Vector Machine model performs better than the Multilayer Neural Networks model.

Statistic	Multilayer Neural Networks	Support Vector Machine
Mean	0.37675	0.35
Variance	0.000109583	0.00019
Degree of freedom	3	
t-stat	2.792994383	
P(T<=t) one tailed	0.034124385	

Table 4: Results of paired t-test

7.2 Performance of learners vs. analysts

To evaluate if the prediction models perform as well as human analysts, we compare the predictions made by the learners and human analysts against the true labels.

First, we selected ten companies from the pool of companies that we used for training. For the selected companies, we obtained the predictions made by analysts from the MarketWatch data source. Then, we used the learners to make buy-sell predictions for these selected companies. We used the sliding window approach discussed in section 6 to get the buy-sell predictions. We combine the prediction results for the years 2002-2005 for these companies. We end up with 120 predicted instances. The prediction of analysts and predictions of the learners are then compared against the true labels to get the level of agreement with the true labels. See table 5.

Inst#	true_label	neural net	svm	analyst
1	BUY	HOLD	HOLD	BUY
2	HOLD	BUY	BUY	HOLD
3	BUY	HOLD	HOLD	BUY
4	HOLD	BUY	BUY	SELL
5	HOLD	HOLD	BUY	HOLD
6	HOLD	BUY	HOLD	HOLD
7	BUY	BUY	BUY	BUY
8	HOLD	HOLD	BUY	SELL
9	HOLD	HOLD	HOLD	HOLD
10	HOLD	HOLD	BUY	HOLD

Table 5: Snapshot of gathered predictions

Agreement was computed using Cohen's Kappa statistic [7]. The kappa statistic is a measure of agreement

between two observers taking into account agreement that could occur by chance. The resulting kappa score of agreement between human analysts and the true labels is 0.48 while the kappa measure between predictions of Support Vector Machines and Neural Networks with the true labels is 0.28 and 0.07 respectively. Table 6 provides a general summary for interpretation of the kappa scores.

Interpretation	Kappa
No agreement	<0
Poor agreement	0.0-0.19
Fair agreement	0.20-0.39
Moderate agreement	0.40-0.59
Substantial agreement	0.60-0.79
Almost perfect agreement	0.80-1.00

Table 6: Interpretation of kappa scores

From the kappa measures obtained, firstly it is import to note that the agreement between human analysts and the true labels is just moderate. This simply means that human predictions are not very reliable as different factors can lead to variation in predictions. The Support Vector Machine model, performs slightly worse than human analysts with a fair agreement with the true labels. This is a good sign, in that the prediction model for Support Vector Machines can perhaps be further improved to make it perform just as well as human analysts. The Neural Networks model on the other hand, performs rather poorly, with a poor agreement with the true labels. This however, could have been due to factors such as insufficient training data or insufficient training time.

With a small amount of training data, we can say that the Support Vector Machine model does reasonably well in making buy-sell predictions. There is still more work to be done in terms of the Neural Networks model as it does not perform anywhere close to human analysts.

7.3 Profitability

The profitability measure is simply to understand whether one would have made more money if they were to buy and sell stocks based on the predictions of our prediction models or with the recommendations of analysts. This analysis is done by "investing" a certain amount in stocks for the ten companies used for testing, described in the previous section.

We assumed that to start off with, we had 40 shares of each company and starting from 2002 up to end of the year 2004 we could sell, buy or hold these shares. Another assumption made was that we have sufficient capital to buy any amount of shares when needed.

According to the recommendations (from experiments in previous section) for a given quarter, shares were sold, bought or held. The price for which they were traded was also used. A point to be noted is that we choose to trade shares in lots of 10 for consistency. At the end of the year the number of shares held by the shareholder was calculated depending on how many were bought or sold in a year. Also at the end of each year the total gain or loss for each quarter was calculated by multiplying the share price with the number of shares traded in each quarter. So ultimately what we observe at the end of the time period is whether we gain or lose on a relative basis. For example for 2002, a loss of \$583 was calculated for the analysts' prediction at the end of the year. The Neural Network model registered a loss of \$1487 and the Support Vector Machine model registered a loss of \$1950 for the same year. Finally, what is noted is the net value of the number of shares held by the shareholder for all the three cases. The cumulative gain or loss for the past years is added and the final number is the net gain or loss (as of 2004 year end in our case).

Surprisingly, what we observe is that the Neural Networks model and Support Vector Machine model yield higher returns as compared to the analysts. A reasonable explanation of this could be that probably the analysts' recommendations did not consider a long term horizon of 3 years i.e. from 2002 to 2004. The analysts were probably focusing on short term gains. This is also observed in the annual relative gain or loss values observed for the analyst.

Essentially what we gauge from this measure is that Neural Networks model somehow promises to register gains in the long run under certain constraints. Even the Support Vector machine model seems to give better returns for a shareholder if he or she follows the predictions based on this model.

8. Discussion

Although there has been considerable amount of efforts in using machine learning for stock price predictions, very little work has been done to explore the possibility of using machine learning to make buy-sell recommendations. This paper provides insights into how machine learning algorithms have been used to make such predictions. We also present ways in which we evaluate the performance of the learners in contrast to human analysts. In this project, we have used Support Vector Machines and Neural Networks learning algorithms.

The paired t-test used to determine if one learner is better than the other shows that the Support Machine model

performs better than the Neural Networks model with a probability of 96.6%. Apart from the paired t-test, having compared the predictions made by the learners and the analysts with the true labels, our conclusion is that with a reasonably sized data, prediction using the Support Vector Machine model is comparable to human analysts. The Neural Networks model however, performs poorly in contrast to the analysts.

We believe that by making further improvements to our current Support Vector Machine and Neural Networks model, we could probably achieve performance as good as human analysts. One suggestion for improvement would be to experiment with the binning approach of feature values to see if this improves the accuracy of predictions. Binning is a way of discretization and involves placing feature values into different bins. For example, for the p/e feature, values of 15-30 go into the "good" bin and p/e value of 0-10 go into the "fair" bin and so on.

Second suggestion would be to increase the size of the training data. This would mean increasing the number of years and the number of companies for which data has been obtained. Finally, the prediction model that we have uses quarterly periodicity. It would be good to experiment with monthly and daily periodicities to see if such predictions are more accurate as opposed to quarterly based predictions. This also means that the features used would be slightly different and should be discriminating monthly or daily features.

Another suggestion could be to increase the number of relevant features and then have different subsets of features to make predictions. It should be kept in mind that the search for features should be based on fundamentals. So there are some features which are tedious to calculate but give a better picture for the company's market position. An example could be efficiently calculating the Free Cash Flows, Enterprise value, etc.

Although the learners that we have used do not perform as well as human analysts in terms of agreement with the true labels, these learners however perform better in terms of profitability. The profitability evaluation shows that the Neural Networks model yields highest returns, followed by the Support Vector Machine model and then the human analysts.

9. References

[1] <http://www.stockneuromaster.com/snm.htm>

- [2] <http://wrds.wharton.upenn.edu/>
- [3] <http://finance.yahoo.com/>
- [4] <http://beginnersinvest.about.com/>
- [5] <http://www.marketwatch.com>
- [6] <http://www.cs.waikato.ac.nz/ml/weka/>
- [7] http://en.wikipedia.org/wiki/Cohen's_kappa
- [8] Karl Nygren, Stock Prediction – A Neural Network Approach, Master Thesis, Royal Institute of Technology, KTH, March 28, 2004
- [9] Ramon Lawrence, Using Neural Networks to forecast stock market prices, Department of Computer Science, University of Manitoba, December 12, 1997
- [10] Jingtao Yao, Chew Lim Tan, Guidelines for Financial Forecasting with Neural Networks, Proceedings of International Conference on Neural Information Processing, Shanghai, China, November 14-18, 2001